

Prompt: Build the Inventory Cleaning Script

You are writing a single Python script, `clean_inventory.py`, that transforms a raw pharmacy-chain inventory export into a star-schema-ready set of files for Power BI. Use pandas. Keep the script modular — one function per stage — and do NOT add abstractions, config frameworks, or CLI options beyond a simple `--input / --output-dir / --category-map` argparse setup. This is a one-shot ETL script that will be re-run monthly on new drops, not a framework. Prefer boring, explicit code over clever code.

Input

- File: `Cleaned_Inventory_Combined.csv` (~215MB, ~1.34M rows)
- Columns: `Report_Month, Account_Name, Store_Code, Store_Name, Pincode, State, City, Product_Code, Product_Name, Category, Manufacturer_Name, Stock_Qty,`

`Stock_Cost_Value, Stock_MRP_Value,`
`Unit_Cost`

- `Report_Month` is text like "May 2026" / "June 2026"
- Multiple rows can share the same `Store_Code` + `Product_Code` + `Report_Month` (batch-level splits) and must be aggregated
- Known issues in the source: ~334 rows with negative `Stock_Qty`, ~88K rows with blank `Pincode`, `Category` mixes real therapy-area values with dosage-form and junk values (e.g. "TABLETS", "GENERAL", "PHARMA" sitting next to "ANTIHYPERTENSIVE MEDICINES")

Steps the script must perform, in order

1. **Load** the CSV in chunks (`chunksize=200_000`) with explicit dtypes
(`Store_Code` , `Product_Code` , `Pincode` as string — never numeric, to avoid silently dropping leading zeros; numeric columns as float32/float64) to keep memory bounded.
2. **Quarantine negative `Stock_Qty` rows** into `quarantined_negative_qty.csv` with all original columns intact. Do not drop them

or net them into the
aggregates silently — a reconciliation step later
depends on knowing exactly
what was pulled out.

3. **Aggregate** the remaining rows to grain
(Report_Month, Account_Name,
Store_Code, Product_Code): sum Stock_Qty ,
Stock_Cost_Value ,
Stock_MRP_Value . Recompute a weighted
 $Unit_Cost = Stock_Cost_Value / Stock_Qty$,
guarding divide-by-zero (return null, don't raise).
4. **Build Dim_Date.csv** : one row per distinct
Report_Month, with Date
(first of month, e.g. 2026-05-01), Month_Name ,
Month_Sort (integer, for
correct chronological sort in Power BI regardless
of alphabetical order).
5. **Build Dim_Store.csv** : one row per Store_Code,
with Store_Name,
Account_Name (chain), State, City, Pincode. For
Pincode: fill blanks using
the most frequent non-null Pincode seen for that
Store_Code anywhere in the
source; leave null only if a store never has one at
all.
6. **Build Dim_Product.csv** : one row per
Product_Code, with Product_Name,
raw Category , Manufacturer_Name . Do NOT

attempt to auto-classify
Category into a clean taxonomy here — that's a
deliberate manual step
(see 7). Auto-classifying a ~78-value taxonomy
that mixes two different
schemes is a business judgment call, not
something to infer silently.

7. **Build `Category_Mapping_Template.csv`** : one row
per distinct raw
Category value, with row count and total
Stock_Cost_Value contribution, and
a blank `Clean_Therapy_Area` column for manual
assignment. If a completed
mapping file is passed via `--category-map` , join
it into Dim_Product as
`Clean_Therapy_Area` instead of regenerating the
template.

8. **Build `Fact_Inventory.csv`** : Store_Code,
Product_Code, Date (join key to
Dim_Date), Stock_Qty, Stock_Cost_Value,
Stock_MRP_Value, Unit_Cost. This is
the aggregated output of step 3 — one row per
Store_Code + Product_Code +
Month.

9. **Reconcile**: assert that
`SUM(Fact_Inventory.Stock_Cost_Value) +`
`SUM(quarantined.Stock_Cost_Value)` equals
the source file's total

Stock_Cost_Value, within float tolerance. Print a clear PASS/FAIL line.

10. **Print a run summary** to stdout: rows in/out at each stage, rows quarantined, Pincodes filled vs. left null, distinct Store/Product/Category counts, before/after value totals.

Output

- All files written to `--output-dir` (default `./output/`), CSV, UTF-8, no index column.

Constraints

- Never modify or overwrite the source CSV.
- Do not attempt category auto-classification (step 7 is manual by design).
- No new dependencies beyond pandas/numpy — must run on a plain Python 3.10+ environment with no extra installs.
- If something in the source doesn't match these assumptions (e.g. a new Report_Month format, an unexpected column),

fail loudly with a clear error
rather than guessing.